

No
01

DOCUMENTATION



mini-baas-infra

Documentation d'Architecture Back-End



DATE

May 28, 2026

EST · MMXXVI



Table des matières

01 Composition observée du projet

1.1 Répartition des rôles proposée

02 Organisation projet

2.1 Méthodologie de travail

2.2 Outils de gestion et communication

03 Stack technique globale

04 Base de données : logique d'ensemble

05 Liste de fonctionnalités implémentées

5.1 Fonctionnalités cœur

06 Modules pédagogiques — proposition de catégorisation

6.1 Modules majeurs (2 pts)

6.2 Modules mineurs (1 pt)

07 1. Liste des Compétences Couvertes

08 2. Expression des Besoins

8.1 2.1 Objectifs du Projet

8.2 Problème à résoudre

8.3 Objectif principal

8.4 Objectifs secondaires

8.5 Public cible

8.6 2.2 Limites du Projet

8.7 Limites techniques actuelles à connaître

8.8 2.3 Livrables Attendus

09 3. Environnement Technique

9.1 3.1 Technologies Front-End

9.2 Pourquoi le front-end n'est pas couplé au dépôt

9.3 3.2 Technologies Back-End

9.4 Bibliothèques significatives

9.5 3.3 Outils & Environnement de Développement

9.6 Architecture de build

10 4. Réalisations Front-End

- 10.1 4.1 Maquettes des Interfaces Utilisateur
- 10.2 Maquette fonctionnelle type — application cliente
- 10.3 Maquette fonctionnelle type — console d'administration
- 10.4 Choix ergonomiques recommandés
- 10.5 Emplacements recommandés pour captures d'écran
- 10.6 4.2 Enchaînement des Maquettes
- 10.7 4.3 Interfaces Statiques
- 10.8 4.4 Partie Dynamique des Interfaces
- 10.9 4.5 Adaptation Web & Web Mobile (Responsive)

11 5. Réalisations Back-End

- 11.1 5.1 Architecture & Structure
- 11.2 Idée directrice
- 11.3 Principe architectural central
- 11.4 Diagramme d'architecture global
- 11.5 Rôle des grandes couches
- 11.6 Monorepo NestJS
- 11.7 Flux d'une requête typique
- 11.8 Pourquoi cette architecture est générique
- 11.9 Lecture académique de l'architecture
- 11.10 Détail service par service
- 11.11 Sous-sections dédiées aux applications NestJS
- 11.12 Sous-sections dédiées aux bibliothèques partagées
- 11.13 Sous-sections dédiées aux services Docker d'infrastructure
- 11.14 Analyse comparative avec un backend sur mesure classique
- 11.15 Exemple de garde d'authentification partagée
- 11.16 Exemple de récupération d'une base enregistrée
- 11.17 Exemple de logique de chiffrement
- 11.18 5.2 Base de Données
- 11.19 Modèle relationnel principal
- 11.20 Schéma logique simplifié
- 11.21 PostgreSQL : isolation par RLS
- 11.22 Tables externes générées automatiquement
- 11.23 MongoDB : modèle documentaire et validateur
- 11.24 Pourquoi le choix polyglotte est pertinent
- 11.25 Registre des bases externes
- 11.26 5.3 API / Routes

11.27 Exemples de payloads

11.28 Pourquoi ces routes sont puissantes

12 6. Sécurité

12.1 Vue d'ensemble des couches de sécurité

12.2 6.1 Mesures de Sécurité Côté Front-End

12.3 6.2 Veille sur les Vulnérabilités

12.4 Vulnérabilités prises en compte

12.5 6.3 Mesures de sécurité côté back-end

12.6 6.4 Pourquoi cette sécurité est adaptée à un backend générique

13 7. Jeu d'Essai

13.1 Vue synthétique du scénario

13.2 Cas nominal

13.3 Cas d'erreur

13.4 Analyse

14 8. Installation & Utilisation

14.1 8.1 Prérequis

14.2 8.2 Installation

14.3 8.3 Lancement du Projet

14.4 Ports et points d'accès usuels

14.5 Ordre de démarrage recommandé pour une démonstration

14.6 8.4 Séquence de démarrage service par service

14.7 8.5 Cycle complet des requêtes principales

15 9. Risques, Limites & Améliorations Futures

15.1 9.1 Risques techniques actuels

15.2 9.2 Limites fonctionnelles

15.3 9.3 Limites pédagogiques à expliciter à l'oral

15.4 9.4 Améliorations futures envisageables

15.5 9.5 Synthèse stratégique

16 10. Glossaire

17 11. Structure du Projet

17.1 Lecture de structure par responsabilités

18 12. Cartographie des Fichiers Sources

18.1 12.1 Cartographie des applications NestJS

- 18.2 12.2 Cartographie des bibliothèques partagées
 - 18.3 12.3 Cartographie des fichiers d'infrastructure majeurs
 - 18.4 12.4 Cartographie documentaire et opérationnelle
-

[19](#) **13. Annexes**

- 19.1 Annexe A – Diagramme complet du cycle d'authentification
 - 19.2 Annexe B – Cycle de création d'un schéma externe
 - 19.3 Annexe C – Cycle de requête universelle
 - 19.4 Annexe D – Extrait de DTO unifié pour le query-router
 - 19.5 Annexe E – Pourquoi ce projet est une architecture back-end généraliste
 - 19.6 Annexe F – Conclusion générale
-

mini-baas-infra — Documentation d'Architecture Back-End

Plateforme BaaS auto-hébergée, orientée Docker Compose, conçue comme une **usine à backends génériques** capable de fournir authentification, données relationnelles, données documentaires, requêtage multi-tenant, temps réel, stockage objet, email transactionnel et politiques de sécurité unifiées sans développement d'API métier spécifique pour chaque nouveau projet.

Team information

Composition observée du projet

À l'état actuel du dépôt, le projet semble être porté principalement par **dlesieur**. Cette documentation est rédigée de manière à pouvoir être utilisée dans un cadre pédagogique individuel **ou** collectif.

Répartition des rôles proposée

Membre	Rôle principal	Responsabilités observées / déduites
dlesieur	Product Owner / Tech Lead / DevOps / Back-End Lead	Conception de l'architecture, intégration Docker, orchestration des services, sécurité, écriture des services NestJS, outillage d'exploitation, documentation technique

Si le projet est présenté en groupe, ce tableau peut être enrichi avec les noms réels de l'équipe. La structure documentaire ci-dessous reste valable.

Organisation projet

Méthodologie de travail

Le dépôt montre une approche **incrémentale et modulaire** :

- construction par services indépendants ;
- montée progressive en complexité ;
- documentation parallèle à l'implémentation ;
- validation régulière par scripts de tests par phases ;
- séparation claire entre infrastructure, services applicatifs, scripts, documentation et configuration.

Cette stratégie est cohérente avec une démarche de type :

- **itérative** pour l'ajout des fonctionnalités ;
- **DevOps** pour l'automatisation du build, du run et des vérifications ;

- **contract-first** pour plusieurs composants exposés via routes et DTOs ;
- **security-first** pour l'authentification, l'isolation et le chiffrement.

Outils de gestion et communication

Même si tous les outils externes ne sont pas visibles dans le dépôt, la structure laisse apparaître une pratique compatible avec :

- Git / GitHub pour le versionnement ;
- Issues / milestones / branches pour la traçabilité ;
- documentation Markdown comme base de communication technique ;
- scripts shell et Makefile comme standard d'exécution pour l'équipe ;
- Docker Compose comme contrat de déploiement partagé.

Stack technique globale

Domaine	Choix	Justification
Orchestration locale	Docker Compose	Reproductibilité, isolement, simplicité de démarrage
Gateway	Kong	Centralisation des plugins, auth, rate-limit, CORS, routage
Auth	GoTrue	Gestion prête à l'emploi des comptes, JWT, MFA, rotation de refresh token
SQL REST	PostgREST	Exposition automatique d'une API relationnelle sans écrire de contrôleurs métier
DB relationnelle	PostgreSQL 16	RLS, robustesse, SQL mature, extensibilité
DB documentaire	MongoDB 7	Modèle souple, change streams, adaptation aux données non strictement relationnelles
Microservices custom	NestJS 10 + TypeScript 5	Modularité, DI, DTO, maintenabilité
Temps réel	realtime-agnostic	Diffusion temps réel des mutations PostgreSQL et MongoDB
Stockage objet	MinIO + AWS SDK S3	Compatibilité S3 et simplicité d'auto-hébergement
Email	Nodemailer + SMTP	Standard simple, interopérable, découplé du fournisseur
Observabilité	Prometheus + Grafana + Loki + Promtail	Mesures, dashboards et logs centralisés
Sécurité des secrets	Vault + chiffrement AES-256-GCM	Réduction de l'exposition des secrets et chiffrement des credentials sensibles

Base de données : logique d'ensemble

Le projet ne repose pas sur **une seule** base de données, mais sur plusieurs couches :

1. **PostgreSQL système** : stocke utilisateurs, rôles, politiques, registres, tables métiers SQL.
2. **MongoDB système** : stocke documents orientés collections avec isolation par `owner_id`.
3. **Bases externes enregistrées** : PostgreSQL ou MongoDB tiers, branchés dynamiquement via l' `adapter-registry`.
4. **Registres internes** : `tenant_databases`, `schema_registry`, `roles`, `user_roles`, `resource_policies`.

Liste de fonctionnalités implémentées

Fonctionnalités cœur

Fonctionnalité	Description
Authentification	Inscription, connexion, JWT, MFA, refresh token rotation
Gateway centralisée	Routage unique, plugins de sécurité, normalisation des échanges
API SQL générique	REST automatique sur PostgreSQL via PostgREST
API Mongo générique	CRUD propriétaire sur collections MongoDB
Requêtage universel	<code>query-router</code> vers PostgreSQL ou MongoDB externes
Registre de bases	Ajout dynamique de bases distantes, credentials chiffrés
Génération de schémas	Création de tables/collections à partir d'une spécification unifiée
Contrôle d'accès	RBAC + ABAC via rôles, politiques, fonction <code>has_permission()</code>
Stockage fichiers	Génération d'URLs présignées S3 compatibles MinIO
Email transactionnel	Envoi SMTP via service dédié
Temps réel	Diffusion des changements de données
Observabilité	Logs, métriques, dashboards
Validation et scripts	Contrôles shell, compose, secrets, smoke tests

Modules pédagogiques — proposition de catégorisation

Cette sous-partie peut être utilisée dans un cadre d'exercice. La pondération ci-dessous est une **proposition de lecture pédagogique** de la complexité du système.

Modules majeurs (2 pts)

Module	Type	Pourquoi majeur
Kong Gateway	Majeur	Point d'entrée unique, sécurité transverse, routage
Auth GoTrue + JWT	Majeur	Identité, émission des tokens, MFA, sécurité utilisateur

Module	Type	Pourquoi majeur
PostgreSQL + PostgREST + RLS	Majeur	Data plane relationnel auto-exposé
MongoDB + mongo-api	Majeur	Data plane documentaire auto-exposé
adapter-registry	Majeur	Fondement du backend générique multi-base
query-router	Majeur	Requêtage universel sans endpoint métier dédié
schema-service	Majeur	Génération de schémas et industrialisation des structures
permission-engine	Majeur	Gouvernance des accès et ABAC

Modules mineurs (1 pt)

Module	Type	Pourquoi mineur
storage-router	Mineur	Important mais périphérique au cœur data/auth
email-service	Mineur	Service transverse simple mais utile
Vault	Mineur	Sécurité d'environnement et gestion des secrets
Observabilité	Mineur	Essentiel en production, indirect fonctionnellement
Trino / Studio / pg-meta / Supavisor	Mineur	Services d'admin, d'exploration ou d'optimisation
Scripts d'automatisation	Mineur	Support de qualité et d'opération

1. Liste des Compétences Couvertes

Ce projet met en œuvre les compétences suivantes issues du référentiel, en les réinterprétant dans un contexte fortement orienté back-end, architecture et intégration.

Activité Type 1 – Développer la partie front-end d'une application web ou web mobile sécurisée

#	Compétence	Mise en œuvre dans le projet
C1	Maquetter des interfaces utilisateur web ou web mobile	Le dépôt ne contient pas de front produit final, mais l'architecture est conçue pour supporter n'importe quel front via API unifiée ; les parcours d'usage sont documentables sous forme de flux d'écrans côté client
C2	Réaliser des interfaces utilisateur statiques web ou web mobile	Le dépôt contient un playground et des conventions d'intégration ; la partie interface finale est volontairement découplée du back-end
C3	Développer la partie dynamique des interfaces utilisateur web ou web mobile	Le système est pensé pour alimenter des interfaces dynamiques grâce à HTTP, JWT, WebSocket et API génériques

Activité Type 2 – Développer la partie back-end d'une application web ou web mobile sécurisée

#	Compétence	Mise en œuvre dans le projet
B1	Concevoir une architecture applicative	Microservices spécialisés, gateway unique, séparation des plans de données
B2	Mettre en place une base de données	PostgreSQL, MongoDB, rôles, politiques, migrations, schémas dynamiques
B3	Développer des composants d'accès aux données	Engines SQL/Mongo, registre d'adaptateurs, query-router
B4	Exposer des services réseau sécurisés	Kong, JWT, key-auth, rate limiting, WAF
B5	Industrialiser et déployer	Docker Compose, bake, Makefile, scripts de validation
B6	Garantir la sécurité	RLS, owner isolation, chiffrement AES-256-GCM, Vault, DTOs, guards

2. Expression des Besoins

2.1 Objectifs du Projet

Le projet répond à un besoin très clair : **disposer d'un backend réutilisable pour n'importe quel produit sans devoir recoder un serveur spécifique pour chaque nouveau cas d'usage.**

Problème à résoudre

Dans de nombreux projets, une part importante du temps est consacrée à reconstruire les mêmes briques :

- authentification ;
- gestion des utilisateurs ;
- stockage relationnel ;
- stockage documentaire ;
- CRUD basiques ;
- contrôle des accès ;
- email ;
- upload de fichiers ;
- logs et observabilité ;
- exposition sécurisée des données.

L'objectif ici est de transformer ces briques répétitives en **moteurs génériques**.

Objectif principal

Construire une plateforme BaaS auto-hébergée qui permette à un front-end, à un produit métier ou à un autre service de consommer des capacités back-end complètes **sans écrire d'API métier de base**.

Objectifs secondaires

- unifier l'authentification derrière un seul point d'entrée ;
- prendre en charge PostgreSQL **et** MongoDB ;
- supporter des bases externes enregistrées dynamiquement ;
- industrialiser la création des schémas ;
- appliquer une sécurité native ;
- fournir une base exploitable localement, en démonstration ou en pré-production ;
- permettre l'observabilité et l'exploitation ;
- rendre le système extensible à d'autres moteurs et services.

Public cible

Cible	Usage
Développeur front-end	Consommer auth, données, temps réel et fichiers sans créer son propre back-end
Équipe produit	Démarrer rapidement un MVP avec des primitives déjà sécurisées
Intégrateur / DevOps	Déployer une plateforme interne de services back-end mutualisés
Équipe pédagogique	Démontrer une architecture moderne, modulaire et sécurisée

2.2 Limites du Projet

Le périmètre est volontairement concentré sur le back-end générique. Le projet **ne couvre pas** directement :

- une application front-end métier finalisée ;
- une logique business spécifique à un domaine fonctionnel précis ;
- une administration visuelle complète de tous les moteurs custom ;
- un cluster Kubernetes de production ;
- une gestion multi-région ;
- un workflow CI/CD complet dans ce dépôt même ;
- la compatibilité avec tous les moteurs SQL/NoSQL existants.

Limites techniques actuelles à connaître

- le moteur principal custom générique cible surtout PostgreSQL et MongoDB ;
- les requêtes universelles restent volontairement encadrées pour garder un bon niveau de sécurité ;
- certaines extensions avancées (fédération, admin UI, objet storage) sont sous profils optionnels ;
- les fronts clients doivent respecter les contrats d'auth, d'API key et de JWT.

2.3 Livrables Attendus

- [x] Infrastructure Docker Compose fonctionnelle
- [x] Passerelle API centralisée
- [x] Authentification prête à l'emploi
- [x] Plan de données SQL générique
- [x] Plan de données MongoDB générique

- [x] Requêtage multi-tenant vers bases externes
- [x] Générateur de schémas SQL/Mongo
- [x] Moteur de permissions RBAC/ABAC
- [x] Documentation technique détaillée
- [x] Scripts de validation et de test
- [x] Support de démonstration technique

3. Environnement Technique

3.1 Technologies Front-End

Ce dépôt est **back-end first**. Il n'embarque pas une interface applicative finale complète, mais il est explicitement conçu pour être consommé par n'importe quel front-end.

Technologie	Version	Rôle
HTML5	-	Consommation possible par n'importe quel client Web
CSS3	-	Hors du dépôt principal, utilisé par tout front connecté au BaaS
JavaScript / TypeScript	Variable selon client	Intégration API, gestion JWT, appels temps réel
React / Vue / Angular / mobile	Non imposé	L'architecture est volontairement agnostique côté client

Pourquoi le front-end n'est pas couplé au dépôt

Le choix architectural fondamental du projet consiste à ne **pas** enfermer la solution dans un front unique. La plateforme doit pouvoir servir :

- une SPA Web ;
- une application mobile ;
- un back-office ;
- un chatbot ;
- un autre microservice.

Autrement dit, le front n'est pas absent par manque, mais parce que le projet vise un rôle d'**infrastructure de backend mutualisée**.

3.2 Technologies Back-End

Technologie	Version	Rôle
TypeScript	5.7.3	Langage principal des microservices NestJS
Node.js	20 (Dockerfile)	Runtime des services custom
NestJS	10.x	Framework modulaire pour les services back-end

Technologie	Version	Rôle
PostgreSQL	16-alpine	Base relationnelle principale
MongoDB	7	Base documentaire principale
Kong	3.8	API Gateway et politique d'accès
GoTrue	v2.188.1	Authentification, JWT, MFA
PostgREST	v12.2.3	API REST automatique sur PostgreSQL
Redis	7-alpine	Cache / support infra
MinIO	RELEASE.2025...	Stockage objet compatible S3
Nodemailer	6.9.x	Envoi d'email SMTP
Prometheus / Grafana / Loki	versions définies dans Compose	Observabilité

Librairies significatives

Librairie	Rôle
<code>class-validator</code>	Validation des DTO
<code>class-transformer</code>	Transformation des requêtes entrantes
<code>nestjs-pino</code>	Logs structurés
<code>@nestjs/axios</code>	Appels HTTP inter-services
<code>pg</code>	Connexion PostgreSQL custom
<code>mongodb</code>	Client MongoDB custom
<code>@aws-sdk/client-s3</code>	Interaction avec MinIO/S3
<code>@willsoto/nestjs-prometheus</code>	Métriques Prometheus pour les services
<code>uuid</code>	Génération d'identifiants et corrélation

3.3 Outils & Environnement de Développement

Outil	Usage
VS Code	Édition et navigation de code
Git / GitHub	Versionnement, historisation, collaboration
Docker / Docker Compose	Exécution locale et orchestrée
Make	Commandes simplifiées d'exploitation
Bash	Scripts de validation, génération de secrets, migrations
Mermaid	Diagrammes d'architecture et de flux
Swagger / OpenAPI	Documentation d'API des services NestJS

Architecture de build

Le dépôt contient un `src/Dockerfile` unifié permettant de builder les 7 applications NestJS via un argument `APP`.

Extrait significatif :

```
ARG APP=adapter-registry

FROM node:${NODE_VERSION}-alpine AS deps
WORKDIR /app
COPY --link package.json package-lock.json ./
RUN npm ci

FROM deps AS build
ARG APP
COPY --link tsconfig.json tsconfig.build.json nest-cli.json ./
COPY --link libs/ ./libs/
COPY --link apps/ ./apps/
RUN npx nest build ${APP}
```

Ce choix évite de maintenir un Dockerfile par microservice, diminue la duplication et renforce la cohérence du monorepo.

4. Réalisations Front-End

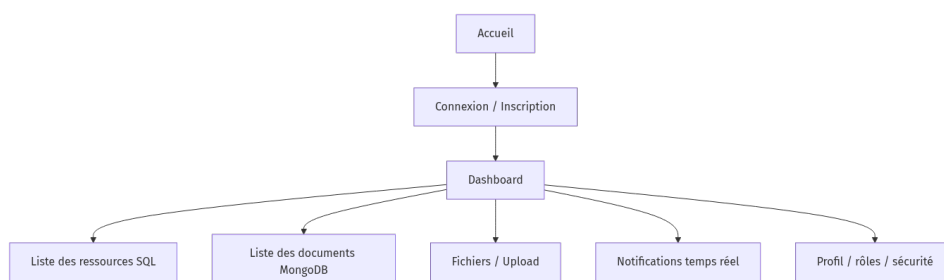
Cette section suit le plan demandé par l'exercice. Le dépôt étant centré sur le back-end, la partie front-end est décrite sous forme de **modèle d'intégration attendu** plutôt que comme un produit visuel final livré dans ce repository.

4.1 Maquettes des Interfaces Utilisateur

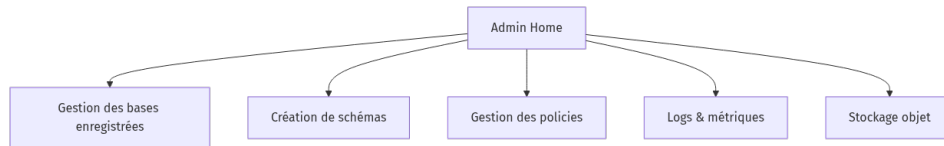
Le système est pensé pour alimenter plusieurs familles d'interfaces :

1. application cliente métier ;
2. back-office d'administration ;
3. console de développement / playground ;
4. application mobile ;
5. microservice consommateur.

Maquette fonctionnelle type — application cliente



Maquette fonctionnelle type — console d'administration



Choix ergonomiques recommandés

- séparation claire entre auth, données, stockage, administration ;
- affichage du contexte utilisateur courant (id, rôle, tenant) ;
- visibilité explicite des permissions ;
- retour d'erreur standardisé ;
- journalisation des requêtes critiques côté console.

Emplacements recommandés pour captures d'écran

Dans une version rendue au jury, il est fortement recommandé d'insérer des captures ou mockups aux emplacements suivants :

1. **écran de connexion / inscription** ;
2. **tableau de bord principal** avec liens vers SQL, MongoDB, schémas, permissions ;
3. **écran d'enregistrement d'une base externe** ;
4. **écran de création d'un schéma** ;
5. **écran de requête universelle** ;
6. **écran d'observabilité** avec métriques ou logs ;
7. **écran de gestion des politiques**.

PLACEHOLDER DE FIGURE — AUTHENTIFICATION

Figure 1 — Écran d'authentification

Insérer ici une capture montrant les champs email, mot de passe, éventuellement MFA, ainsi que le parcours de récupération / inscription.

PLACEHOLDER DE FIGURE — DASHBOARD

Figure 2 — Tableau de bord de la plateforme

Insérer ici une capture de l'écran principal présentant les modules auth, SQL, Mongo, schemas, storage, permissions et monitoring.

PLACEHOLDER DE FIGURE — REGISTRE DE BASES

Figure 3 — Interface d'enregistrement d'une base externe

Insérer ici une capture du formulaire permettant de choisir le moteur, le nom logique et la chaîne de connexion.

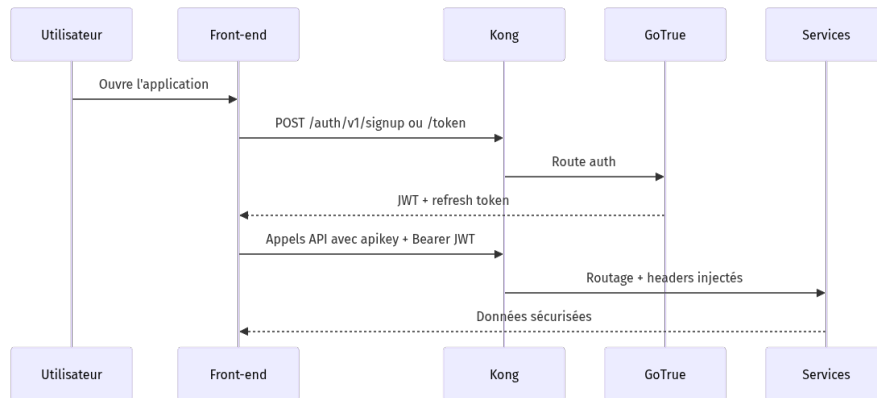
PLACEHOLDER DE FIGURE — OBSERVABILITÉ

Figure 4 — Dashboard d'observabilité

Insérer ici une capture Grafana ou Prometheus illustrant l'état du système et la supervision.

4.2 Enchaînement des Maquettes

Le back-end est construit pour supporter le cycle utilisateur suivant :



4.3 Interfaces Statiques

L'interface statique recommandée pour consommer le projet peut être structurée autour de blocs simples :

```

<header class="topbar">
  <h1>Mini BaaS Console</h1>
  <div class="session">
    <span id="user-email"></span>
    <button id="logout">Se déconnecter</button>
  </div>
</header>

<main class="layout">
  <aside class="sidebar">
    <button data-view="sql">SQL</button>
    <button data-view="mongo">MongoDB</button>
    <button data-view="schemas">Schemas</button>
    <button data-view="policies">Policies</button>
    <button data-view="storage">Storage</button>
  </aside>

  <section class="content" id="app-view"></section>
</main>

```

Extrait CSS possible :


```

.layout {
  display: grid;
  grid-template-columns: 280px 1fr;
  min-height: 100vh;
}

.sidebar {
  border-right: 1px solid #e5e7eb;
  padding: 1rem;
}

.content {
  padding: 1.5rem;
  background: #fafafa;
}

```

4.4 Partie Dynamique des Interfaces

Le front-end typique exploite principalement :

- les appels `fetch` / `axios` ;
- le stockage temporaire du JWT ;
- l'ajout des headers `apikey` et `Authorization` ;
- les WebSockets pour le temps réel ;
- l'affichage des réponses paginées.

Exemple d'appel dynamique vers le `query-router` :

```

async function queryExternalDatabase(dbId, table, token, apikey, payload) {
  const res = await fetch(`/query/v1/query/${dbId}/tables/${table}`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
      apikey,
    },
    body: JSON.stringify(payload),
  });

  if (!res.ok) {
    throw new Error(`HTTP ${res.status}`);
  }

  return res.json();
}

```

4.5 Adaptation Web & Web Mobile (Responsive)

Le système est particulièrement compatible avec une approche responsive car la logique métier est portée côté API et non côté rendu. Le front n'a donc qu'à gérer :

- la session ;
- les formulaires ;
- l'affichage des données ;
- l'état de chargement ;
- les erreurs.

Exemple de règles responsive :

```
@media (max-width: 768px) {
  .layout {
    grid-template-columns: 1fr;
  }

  .sidebar {
    border-right: 0;
    border-bottom: 1px solid #e5e7eb;
  }
}
```

5. Réalisations Back-End

5.1 Architecture & Structure

Le projet suit une architecture **microservices intégrés par gateway**, avec une forte orientation **backend factory**.

Idée directrice

Le système n'est pas un back-end d'application unique, mais un **socle** capable d'alimenter plusieurs projets différents grâce à des primitives génériques.

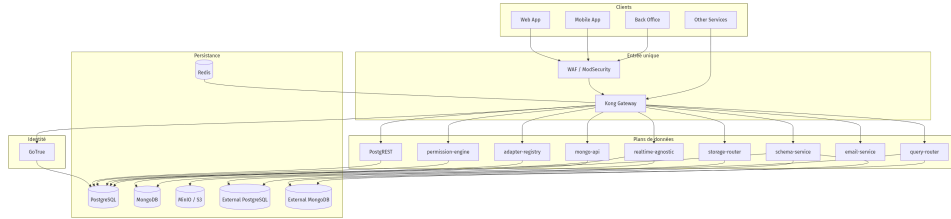
Principe architectural central

Au lieu de créer pour chaque projet :

- un service auth ;
- un service users ;
- un CRUD SQL ;
- un CRUD NoSQL ;
- un moteur d'upload ;
- un système de politiques ;
- un service email ;

...le dépôt construit une **plateforme partagée** déjà prête.

Diagramme d'architecture global



Rôle des grandes couches

Couche	Rôle
Ingress	WAF + Kong
Identity	GoTrue
Data plane SQL	PostgREST + PostgreSQL
Data plane document	mongo-api + MongoDB
Data plane externe	adapter-registry + query-router + schema-service
Policy plane	permission-engine
Side services	storage-router, email-service, realtime
Ops plane	Prometheus, Grafana, Loki, Vault, Makefile, scripts

Monorepo NestJS

Le monorepo `src/` contient :

- 7 applications ;
- 3 bibliothèques partagées.

APPLICATIONS

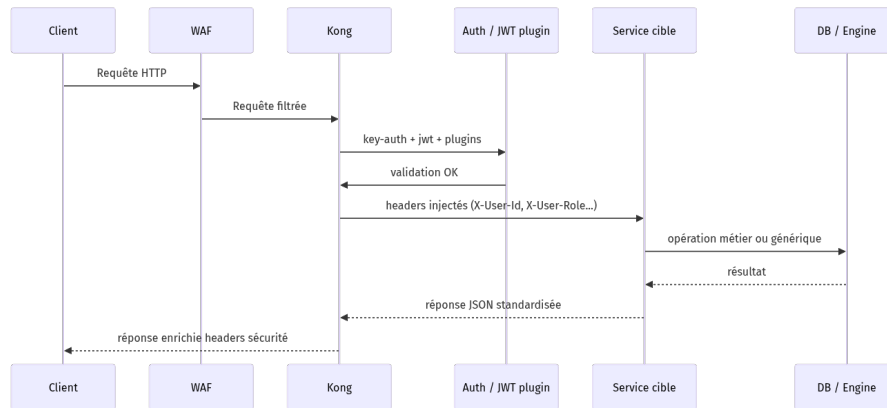
Application	Rôle
<code>adapter-registry</code>	stocker les connexions de bases externes chiffrées
<code>query-router</code>	exécuter des opérations génériques sur bases enregistrées
<code>schema-service</code>	créer / supprimer tables ou collections sur bases enregistrées
<code>permission-engine</code>	gérer rôles et politiques ABAC/RBAC
<code>mongo-api</code>	CRUD générique MongoDB local
<code>storage-router</code>	générer des URLs présignées
<code>email-service</code>	envoyer des emails SMTP

BIBLIOTHÈQUES

Bibliothèque	Rôle
<code>libs/common</code>	guards, decorators, filters, pipes, interfaces

Bibliothèque	Rôle
libs/database	services PostgreSQL et MongoDB partagés
libs/health	endpoints live/ready standardisés

Flux d'une requête typique



Pourquoi cette architecture est générique

Le caractère générique vient de plusieurs choix structurants :

1. les routes ne codent pas un domaine métier précis ;
2. les schémas peuvent être créés dynamiquement ;
3. les bases externes sont enregistrables à chaud ;
4. les contrôles d'accès sont pilotés par données ;
5. le front consomme uniquement des primitives standards ;
6. les services sont substituables et découplés.

Autrement dit, le dépôt agit comme un **back-end programmable par configuration, schéma et policies**, et non comme un code d'application ad hoc.

Lecture académique de l'architecture

Dans une lecture plus universitaire, cette plateforme peut être comprise comme la combinaison de quatre plans complémentaires :

1. **plan d'identité** : authentifie les utilisateurs et produit les preuves d'identité ;
2. **plan de contrôle** : décide qui peut faire quoi ;
3. **plan de données** : exécute effectivement les opérations sur les bases ;
4. **plan d'exploitation** : observe, déploie, valide et maintient la plateforme.

Cette séparation est importante car elle évite de confondre :

- la preuve d'identité,
- la décision d'autorisation,
- la manipulation de données,
- et la gouvernance opérationnelle.

Du point de vue de l'ingénierie logicielle, cette approche améliore :

- la **maintenabilité**, car chaque service a une responsabilité claire ;

- la **scalabilité organisationnelle**, car plusieurs équipes peuvent intervenir par domaine ;
- la **réutilisabilité**, car les primitives restent indépendantes d'un métier ;
- la **testabilité**, car chaque couche peut être validée séparément.

Détail service par service

1. KONG GATEWAY

Kong est le **point d'entrée unique**.

Fonctions assurées :

- routage ;
- plugin `key-auth` ;
- plugin `jwt` ;
- `cors` ;
- `correlation-id` ;
- `response-transformer` pour les headers de sécurité ;
- `rate-limiting` ;
- `request-size-limiting` ;
- `ip-restriction` sur certaines routes d'administration ;
- plugin `pre-function` pour propager les claims JWT aux services en headers de confiance.

Extrait important du comportement : le JWT est vérifié par Kong, puis ses claims sont projetés en :

- `X-User-Id`
- `X-User-Email`
- `X-User-Role`
- `X-Request-ID`

Les services NestJS n'ont donc pas besoin de revalider eux-mêmes le JWT pour le parcours standard via gateway.

2. GOTRUE

GoTrue fournit :

- signup ;
- login ;
- JWT ;
- MFA/TOTP ;
- refresh token rotation ;
- OAuth externes configurables ;
- email auth.

Il constitue la **source d'identité** du système.

3. POSTGRESQL + POSTGRES

Ce binôme expose automatiquement une API relationnelle. Le choix est très fort : plutôt que coder des contrôleurs CRUD spécifiques, on laisse PostgREST générer l'API à partir des tables et des droits SQL.

La sécurité est garantie par les politiques RLS.

4. MONGODB + MONGO-API

MongoDB n'a pas un équivalent natif de PostgREST. Le projet fournit donc un service custom `mongo-api` qui implémente un CRUD générique owner-scoped.

Caractéristiques :

- `owner_id` injecté automatiquement ;
- filtres nettoyés ;
- `_id` validé ;
- pagination ;
- tri ;
- métriques Prometheus ;
- opérations d'admin séparées.

5. ADAPTER-REGISTRY

Service central pour la **connexion dynamique à des bases externes**.

Fonctions :

- enregistrer une base PostgreSQL ou MongoDB ;
- chiffrer sa chaîne de connexion ;
- appliquer une isolation multi-tenant via RLS sur `tenant_databases` ;
- permettre aux services internes autorisés de récupérer le secret déchiffré.

Sans lui, il n'y a pas de backend factory multi-projet.

6. QUERY-ROUTER

Le `query-router` est le moteur universel d'accès aux bases enregistrées.

Il :

- demande la connexion au registre ;
- détecte le moteur ;
- délègue vers l'engine PostgreSQL ou MongoDB ;
- impose l'isolation utilisateur ;
- applique les limites de requête.

Il permet donc d'interroger **n'importe quelle base enregistrée** sans écrire d'endpoint sur mesure.

7. SCHEMA-SERVICE

Le `schema-service` industrialise la création des structures.

Il prend une spécification unifiée (`name` , `engine` , `database_id` , `columns` , etc.), puis :

- crée une table PostgreSQL avec colonnes standardisées et RLS ;
- ou crée une collection MongoDB avec validateur JSON Schema ;
- enregistre le résultat dans `schema_registry` .

Ce service est ce qui rapproche le plus le projet d'un **backend factory** au sens fort : on ne crée plus les structures à la main dans chaque projet.

8. PERMISSION-ENGINE

Le moteur de permissions gère :

- les rôles ;
- les rôles utilisateur ;
- les politiques ressource/action ;
- la fonction SQL `has_permission()` .

Il complète le RLS :

- le RLS protège les lignes ;
- le permission-engine gouverne les accès applicatifs plus généraux.

9. STORAGE-ROUTER

Ce service génère des URLs S3 présignées. Il évite d'exposer des credentials de stockage au client.

Il applique aussi une isolation simple par préfixage du chemin avec l'identifiant utilisateur.

10. EMAIL-SERVICE

Service dédié à l'envoi d'emails SMTP : découplage du transport, maintien d'une responsabilité claire et mutualisation entre projets.

11. REALTIME

Service de diffusion des changements temps réel. Il complète les opérations CRUD en permettant à un front de réagir aux mutations de données sans polling permanent.

Sous-sections dédiées aux applications NestJS

Les paragraphes précédents donnent une vue synthétique. Pour une lecture de soutenance, il est utile d'aller plus loin application par application.

ADAPTER-REGISTRY — REGISTRE SÉCURISÉ D'ADAPTATEURS

Responsabilité principale : faire le lien entre un utilisateur / tenant et une base de données externe enregistrée dynamiquement.

Capacités fonctionnelles :

- enregistrer un moteur (`postgresql` , `mongodb` , etc.) ;
- stocker le nom logique d'une base ;
- chiffrer la chaîne de connexion ;
- lister les bases visibles par le tenant courant ;
- fournir le secret déchiffré aux services internes autorisés ;
- supprimer une entrée de registre.

Choix techniques notables :

- chiffrement AES-256-GCM ;
- table `tenant_databases` protégée par RLS ;
- `ServiceTokenGuard` pour l'accès interne au endpoint de connexion ;
- exposition Swagger ;
- usage de `PostgresService` partagé.

Intérêt dans l'architecture : c'est la brique qui rend la plateforme multi-projet et multi-base. Sans elle, le système resterait limité aux bases internes.

QUERY-ROUTER — MOTEUR UNIVERSEL DE REQUÊTES

Responsabilité principale : offrir un point d'accès unique pour exécuter des opérations génériques sur une base enregistrée.

Capacités fonctionnelles :

- résolution de la connexion réelle ;
- dispatch moteur SQL / Mongo ;
- opérations CRUD normalisées ;
- limitation des résultats ;
- support du filtrage et du tri ;
- propagation du contexte utilisateur pour l'isolation.

Choix techniques notables :

- moteur PostgreSQL avec validation des noms de table / colonne ;
- moteur MongoDB avec nettoyage des filtres ;
- injection automatique de `owner_id` sur insertion ;
- propagation de `app.current_user_id` pour les tables externes RLS.

Intérêt dans l'architecture : il supprime la nécessité d'écrire un service d'accès aux données pour chaque base externe et chaque projet.

SCHEMA-SERVICE — INDUSTRIALISATION DES STRUCTURES

Responsabilité principale : transformer une spécification de schéma unifiée en structure physique réelle sur un moteur PostgreSQL ou MongoDB.

Capacités fonctionnelles :

- création de table PostgreSQL ;
- activation RLS sur base externe ;
- création de collection MongoDB ;
- pose d'un validateur JSON Schema ;
- suppression de structures ;
- enregistrement du schéma produit dans `schema_registry`.

Choix techniques notables :

- ajout systématique de colonnes techniques ;
- garde-fou sur les types SQL supportés ;
- création d'une fonction `current_user_id()` pour les bases externes PostgreSQL ;
- persistance d'un registre central de schémas créés.

Intérêt dans l'architecture : il transforme le dépôt en fabrique de structures de données, et pas seulement en passerelle HTTP.

PERMISSION-ENGINE — GOUVERNANCE ET ABAC

Responsabilité principale : exprimer les droits sous forme de rôles et de politiques de manière centralisée.

Capacités fonctionnelles :

- affectation de rôles ;
- révocation ;
- consultation des rôles d'un utilisateur ;
- création / suppression de politiques ;
- évaluation d'un accès via `has_permission()`.

Choix techniques notables :

- stockage SQL des rôles et politiques ;
- hiérarchie par priorité ;
- logique deny-first à priorité égale ;
- articulation avec `RolesGuard` côté API.

Intérêt dans l'architecture : il évite que les règles d'autorisation soient dispersées dans chaque microservice ou application cliente.

MONGO-API — DATA PLANE DOCUMENTAIRE GÉNÉRIQUE

Responsabilité principale : exposer un CRUD propre sur MongoDB local sans code métier spécifique.

Capacités fonctionnelles :

- création de documents ;
- lecture paginée et filtrée ;
- lecture par identifiant ;
- mise à jour partielle ;
- suppression ;
- endpoints d'administration de schémas / index.

Choix techniques notables :

- injection de `owner_id` ;
- suppression des champs interdits ;
- validation des ObjectId ;
- instrumentation Prometheus ;
- séparation entre `collections` et `admin`.

Intérêt dans l'architecture : il fournit à MongoDB ce que PostgREST fournit à PostgreSQL : une couche API générique stable.

STORAGE-ROUTER — FAÇADE D'ACCÈS AU STOCKAGE OBJET

Responsabilité principale : sécuriser l'accès au stockage par génération d'URLs temporaires.

Capacités fonctionnelles :

- signature GET ;
- signature PUT ;
- délai d'expiration contrôlé ;
- préfixage automatique du chemin avec l'utilisateur.

Intérêt dans l'architecture : il retire au front la responsabilité de manipuler des secrets d'accès S3.

EMAIL-SERVICE — SERVICE TRANSVERSE D'ENVOI

Responsabilité principale : centraliser les envois SMTP et découpler les applications métiers du transport email.

Capacités fonctionnelles :

- envoi HTML ou texte ;
- configuration SMTP externe ;
- vérification de connectivité ;
- journalisation structurée.

Intérêt dans l'architecture : mutualiser l'email évite de recoder la même logique dans chaque projet consommateur.

Sous-sections dédiées aux bibliothèques partagées

LIBS/COMMON

Cette bibliothèque mutualise le vocabulaire de sécurité et de transport : `AuthGuard`, `RolesGuard`, `ServiceTokenGuard`, décorateur `CurrentUser`, filtre d'exceptions, pipe de validation, intercepteur de corrélation.

Apport architectural : elle évite la divergence de comportement entre services.

LIBS/DATABASE

Cette bibliothèque fournit les services `PostgresService` et `MongoService`.

Apport architectural : centralisation des pools, de la santé des connexions, et des patterns d'accès.

LIBS/HEALTH

Cette bibliothèque standardise les endpoints `live` et `ready`.

Apport architectural : facilite l'orchestration Compose, les healthchecks et l'observabilité.

Sous-sections dédiées aux services Docker d'infrastructure

Au-delà des applications NestJS, l'architecture repose sur plusieurs conteneurs d'infrastructure qui méritent eux aussi une lecture détaillée.

WAF

Ce conteneur joue le rôle de bouclier HTTP initial. Il complète Kong par une logique de défense plus orientée filtrage de patterns suspects.

KONG

Kong est le centre nerveux du trafic HTTP. Son fichier déclaratif définit :

- les routes ;
- les services upstream ;
- les plugins globaux ;
- les règles spécifiques à certains endpoints.

POSTGRES

Base relationnelle principale du système. Elle porte à la fois les données de démonstration et les métadonnées critiques d'architecture.

DB-BOOTSTRAP

Job one-shot de préparation :

- rôles SQL ;
- schéma auth ;
- tables de démonstration ;
- politiques RLS ;
- tables de registre ;
- permissions de base.

Son existence renforce le caractère reproductible de la plateforme.

MONGO + MONGO-INIT

MongoDB est démarré avec un replica set afin de supporter les change streams. `mongo-init` finalise l'initialisation.

GOTRUE

Fournisseur d'identité du système. Il remplace le besoin d'un développement sur mesure pour toute la partie comptes, login et MFA.

POSTGREST

Expose la donnée relationnelle directement depuis PostgreSQL. C'est un accélérateur considérable pour construire un backend sans re-développer les opérations basiques.

REALTIME

Diffuse les changements de données. C'est la brique qui rapproche la plateforme d'une expérience moderne orientée événements.

REDIS

Service d'appui pour la pile, utile pour certains patterns de cache ou composants temps réel selon les scénarios d'usage.

MINIO

Fournit un stockage objet auto-hébergé compatible S3, activable en profil `extras`.

VAULT + VAULT-INIT

Participent à la stratégie globale de gestion des secrets. Même si le projet utilise aussi un `.env` pour le local, l'architecture montre une orientation vers une gestion plus mature des secrets.

PROMETHEUS , GRAFANA , LOKI , PROMTAIL

Ils constituent le plan observabilité :

- métriques ;
- tableaux de bord ;
- centralisation des logs.

STUDIO , PG-META , SUPAVISOR , TRINO

Services complémentaires d'administration, de pooling et d'exploration. Ils ne sont pas obligatoires au cœur du backend, mais renforcent l'écosystème de la plateforme.

Analyse comparative avec un backend sur mesure classique

Pour mieux comprendre l'originalité du projet, il est utile de le comparer à une architecture plus classique dans laquelle une équipe construit manuellement son API métier.

Critère	Backend classique spécifique	mini-baas-infra
Authentification	Souvent développée ou intégrée à la main	Mutualisée via GoTrue
CRUD SQL	Contrôleurs et services spécifiques	Automatique via PostgREST
CRUD Mongo	Souvent ad hoc par collection	Générique via <code>mongo-api</code>
Gestion des bases externes	Rare, coûteuse, spécifique	Native via <code>adapter-registry</code>
Création de schémas	Scripts ou migrations dédiés par projet	Générique via <code>schema-service</code>
Permissions	Souvent dispersées dans le code	Centralisées via <code>permission-engine</code>
Stockage	Intégration par projet	Mutualisé via <code>storage-router</code>
Email	Implémentation spécifique	Mutualisé via <code>email-service</code>
Réutilisabilité	Faible à moyenne	Élevée
Time-to-market	Plus long	Réduit

Cette comparaison montre que le projet ne se contente pas d'être une stack technique : il réduit explicitement la redondance structurelle entre projets.

Exemple de garde d'authentification partagée

```
canActivate(context: ExecutionContext): boolean {
  const req = context.switchToHttp().getRequest<Request>();
  const userId = req.headers['x-user-id'] as string | undefined;

  if (!userId) {
    throw new UnauthorizedException('Missing authenticated user headers');
  }

  req.user = {
    id: userId,
    email: (req.headers['x-user-email'] as string | undefined) ?? '',
    role: (req.headers['x-user-role'] as string | undefined) ?? 'authenticated',
  };

  return true;
}
```

Ce code montre bien la philosophie du projet : le service fait confiance au gateway, ce qui simplifie fortement les microservices.

Exemple de récupération d'une base enregistrée

```
private async fetchConnection(dbId: string, userId: string):
Promise<AdapterResponse> {
  const url = `${this.registryUrl}/databases/${dbId}/connect`;
  const { data } = await firstValueFrom(
    this.http.get<AdapterResponse>(url, {
      headers: {
        'X-Service-Token': this.serviceToken,
        'X-Tenant-Id': userId,
      },
    }),
  );
  return data;
}
```

Le `query-router` n'embarque aucun secret en dur pour les bases externes : il passe toujours par le registre.

Exemple de logique de chiffrement

Le registre chiffre les chaînes de connexion avec AES-256-GCM et dérivation `scrypt`.

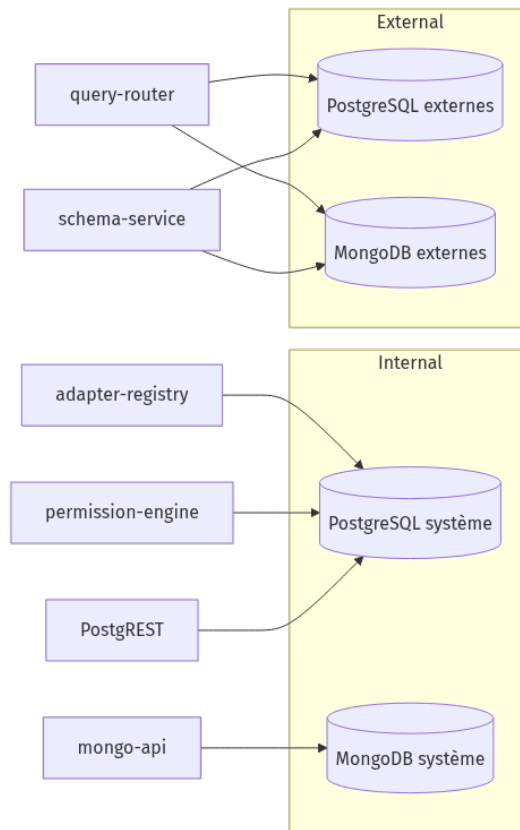
Pourquoi ce choix ?

- confidentialité au repos ;
- authenticité de la donnée via tag GCM ;

- robustesse cryptographique ;
- adaptation naturelle à un stockage binaire en base.

5.2 Base de Données

La stratégie base de données est **polyglotte**.



Modèle relationnel principal

Le PostgreSQL système contient plusieurs familles de tables.

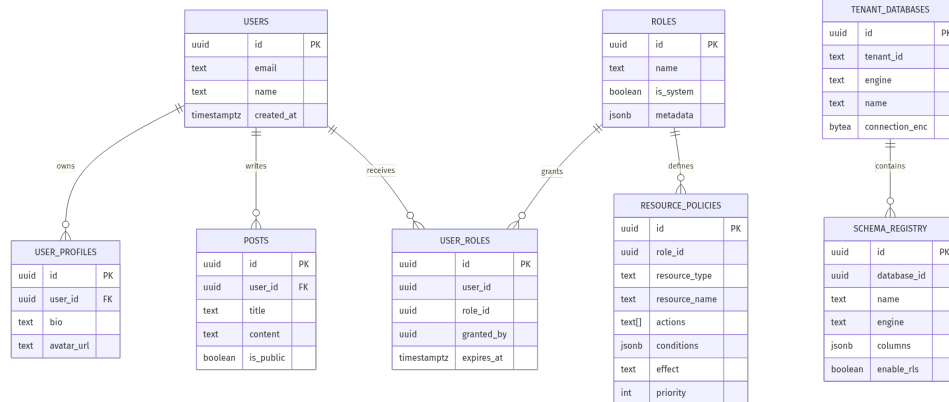
TABLES MÉTIER / DÉMONSTRATION

- `users`
- `user_profiles`
- `posts`
- `projects`
- `mock_orders`

TABLES D'INFRASTRUCTURE

- `tenant_databases`
- `schema_registry`
- `roles`
- `user_roles`
- `resource_policies`
- `schema_migrations`

Schéma logique simplifié



PostgreSQL : isolation par RLS

L'isolation est déléguée à la base elle-même.

Exemple simplifié :

```

CREATE POLICY projects_owner_crud ON public.projects
FOR ALL USING (
  auth.uid()::text = owner_id
)
WITH CHECK (
  auth.uid()::text = owner_id
);

```

Sur les bases externes créées par `schema-service`, la logique est adaptée pour utiliser le contexte injecté via `current_setting('app.current_user_id')`.

Tables externes générées automatiquement

Quand `schema-service` crée une table PostgreSQL externe, il ajoute de façon standard :

- `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`
- `owner_id UUID NOT NULL`
- `created_at TIMESTAMPTZ DEFAULT now()`
- `updated_at TIMESTAMPTZ DEFAULT now()`

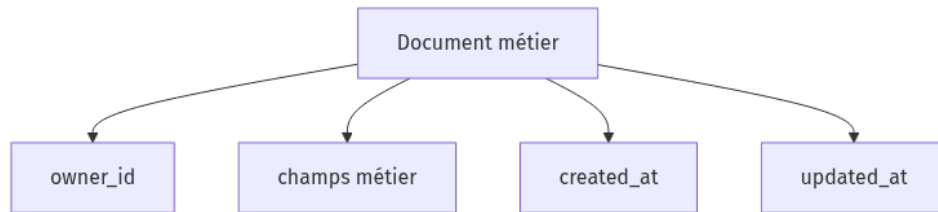
Ce choix permet d'imposer un socle commun à tous les projets sans exiger que chaque développeur repense sa stratégie d'isolation.

MongoDB : modèle documentaire et validateur

La logique MongoDB repose sur :

- un `owner_id` obligatoire ;
- des timestamps ;
- des validateurs JSON Schema ;
- un CRUD filtré côté API.

Schéma logique type :



Pourquoi le choix polyglotte est pertinent

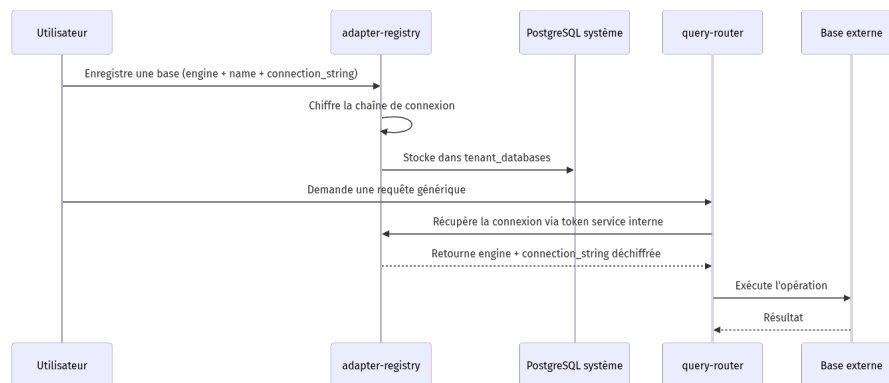
Besoin	PostgreSQL	MongoDB
Données relationnelles strictes	Excellent	Moyen
Schéma souple	Moyen	Excellent
RLS natif	Excellent	Non natif
APIs auto-générées	Excellent avec PostgREST	Nécessite service custom
Change streams / temps réel	Oui via mécanismes dédiés	Excellent

Le projet ne force donc pas un modèle unique. Il choisit le bon moteur selon la nature des données.

Registre des bases externes

Le backend generic factory prend tout son sens avec `tenant_databases`.

Processus :



5.3 API / Routes

L'API du projet est volontairement structurée par domaines transverses plutôt que par logique métier spécifique.

ROUTES PRINCIPALES EXPOSÉES VIA KONG

Méthode	Route externe	Service	Description	Auth require
POST	<code>/auth/v1/signup</code>	GoTrue	inscription utilisateur	apikey
POST	<code>/auth/v1/token</code>	GoTrue	connexion et JWT	apikey

Méthode	Route externe	Service	Description	Auth requise
GET	/rest/v1/...	PostgreSQL	accès REST aux tables PostgreSQL	apikey + JWT selon ressource
GET/POST/PATCH/DELETE	/mongo/v1/collections/:name/documents	mongo-api	CRUD générique MongoDB	apikey + JWT
GET	/mongo/v1/admin/collections	mongo-api	liste des collections	apikey + JWT
POST	/query/v1/query/:dbId/tables/:table	query-router	requête universelle base enregistrée	apikey + JWT
GET	/query/v1/query/:dbId/tables	query-router	liste des tables / collections d'une base enregistrée	apikey + JWT
POST	/schemas/v1/schemas	schema-service	création table / collection	apikey + JWT
GET	/schemas/v1/schemas	schema-service	liste des schémas enregistrés	apikey + JWT
DELETE	/schemas/v1/schemas/:id	schema-service	suppression d'un schéma	apikey + JWT
POST	/admin/v1/databases	adapter-registry	enregistrement d'une base externe	apikey + JWT
GET	/admin/v1/databases	adapter-registry	liste des bases de l'utilisateur	apikey + JWT
POST	/permissions/v1/permissions/check	permission-engine	vérification d'accès	apikey + JWT
GET	/permissions/v1/policies	permission-engine	lecture des politiques	apikey + JWT admin
POST	/storage/v1/sign/:bucket/*	storage-router	URL présignée	apikey + JWT
POST	/email/v1/send	email-service	envoi d'email	apikey + JWT
GET	/realtime/v1/...	realtime	temps réel	apikey + JWT

Exemples de payloads

ENREGISTREMENT D'UNE BASE EXTERNE

```
{
  "engine": "postgresql",
  "name": "customer-crm",
  "connection_string": "postgres://user:pass@host:5432/crm"
}
```

CRÉATION D'UN SCHÉMA POSTGRESQL

```
{
  "name": "contacts",
  "engine": "postgresql",
  "database_id": "550e8400-e29b-41d4-a716-446655440000",
  "enable_ri": true,
  "columns": [
    { "name": "first_name", "type": "text", "nullable": false },
    { "name": "last_name", "type": "text", "nullable": false },
    { "name": "email", "type": "text", "nullable": false, "unique": true }
  ]
}
```

REQUÊTE UNIVERSELLE MONGODB

```
{
  "action": "find",
  "filter": { "status": "active" },
  "sort": { "created_at": "desc" },
  "limit": 20,
  "offset": 0
}
```

REQUÊTE UNIVERSELLE POSTGRESQL

```
{
  "action": "insert",
  "data": {
    "title": "First note",
    "content": "Generic backend factory demo"
  }
}
```

Pourquoi ces routes sont puissantes

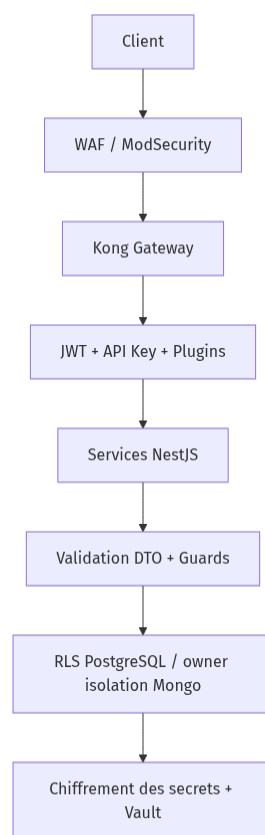
Parce qu'elles reposent sur des **moteurs** et non sur des cas métiers figés :

- le même endpoint sert à plusieurs tables ;
- le même service fonctionne pour plusieurs projets ;
- la logique de sécurité reste uniforme ;
- la maintenance se concentre sur les primitives, pas sur des doublons.

6. Sécurité

La sécurité n'est pas un ajout tardif dans ce dépôt : elle est un **axe d'architecture**.

Vue d'ensemble des couches de sécurité



6.1 Mesures de Sécurité Côté Front-End

Même si le front n'est pas dans ce dépôt, les règles de sécurité côté client sont clairement induites par l'architecture.

Vulnérabilité	Mesure mise en place	Application au projet
XSS	Éviter <code>innerHTML</code> , utiliser échappement et rendu sûr	Tout client consommant les APIs doit afficher les données de manière sûre
CSRF	Usage de JWT Bearer et <code>apikey</code> plutôt que cookies de session implicites	Les appels sont explicitement authentifiés

Vulnérabilité	Mesure mise en place	Application au projet
Vol de token	Stockage prudent côté client, durée de vie limitée, refresh token rotation	Géré par GoTrue et le client
Exposition de secrets	Aucun secret DB ni SMTP ni S3 côté front	Toute opération sensible passe par les services
Accès latéral	Headers et claims gérés par Kong, pas par le client directement	Le front ne forge pas sa propre identité
Upload dangereux	URL présignée générée côté serveur	Le client n'obtient pas les credentials S3

6.2 Veille sur les Vulnérabilités

Sources de référence pertinentes pour ce projet :

- OWASP Top 10 ;
- documentations Kong, PostgreSQL, MongoDB, GoTrue ;
- guides de sécurité Docker ;
- avis CVE des images utilisées ;
- bonnes pratiques Node.js / NestJS.

Vulnérabilités prises en compte

Vulnérabilité	Risque	Réponse dans l'architecture
Bypass d'auth	Accès non autorisé aux services	Kong applique <code>key-auth</code> et <code>jwt</code>
Falsification des claims	Usurpation d'identité	Les headers utilisateur proviennent du gateway, pas du client
Injection SQL	Altération de requêtes	Validation des noms, usage de paramètres, opérations encadrées
Injection Mongo	Filtres dangereux	Suppression de <code>\$where</code> , nettoyage des clés sensibles
Escalade latérale entre tenants	Fuite de données	RLS SQL, <code>owner_id</code> Mongo, politiques
Fuite des chaînes de connexion	Compromission de bases externes	AES-256-GCM + stockage chiffré
Bruteforce / abus d'API	Déni de service	Rate-limits Kong
Upload malveillant	Contenus abusifs	Présignature contrôlée et isolation par préfixe utilisateur
Sur-exposition admin	Accès back-office	<code>ip-restriction</code> , rôles, clés de service

6.3 Mesures de sécurité côté back-end

1. WAF EN FRONTAL

Le service `waf` ajoute une première barrière contre certaines classes d'attaques HTTP connues.

2. GATEWAY AVEC POLITIQUES GLOBALES

Kong applique :

- contrôle d'API key ;
- validation JWT ;
- en-têtes de sécurité ;
- corrélation de requêtes ;
- taille de payload ;
- rate limiting.

3. VALIDATION STRICTE DES ENTRÉES

Les services NestJS utilisent `class-validator` et `class-transformer` via un pipe global.

4. GUARDS ET RÔLES

- `AuthGuard` ;
- `RolesGuard` ;
- `ServiceTokenGuard` .

5. ISOLATION DES DONNÉES

- PostgreSQL : `ROW LEVEL SECURITY` ;
- MongoDB : filtrage systématique par `owner_id` dans les services custom ;
- bases externes : contexte `app.current_user_id` propagé pour les tables générées.

6. CHIFFREMENT DES SECRETS SENSIBLES

Les chaînes de connexion externes ne sont jamais stockées en clair.

7. SECRETS D'ENVIRONNEMENT

Le script `generate-env.sh` fabrique un `.env` avec secrets aléatoires pour le développement local.

Extrait représentatif :

```
JWT_SECRET="$(openssl rand -hex 32)"
VAULT_ENC_KEY="$(openssl rand -hex 16)"
MINIO_ROOT_PASSWORD="$(openssl rand -hex 16)"
```

8. VÉRIFICATION AUTOMATISÉE

Le script `validate-all.sh` exécute :

- vérification shell ;
- vérification JavaScript ;
- validation Docker Compose ;
- contrôle des secrets ;

- scan de secrets hardcodés.

6.4 Pourquoi cette sécurité est adaptée à un backend générique

Une architecture générique est plus exposée qu'un back-end métier fermé, car elle manipule des primitives réutilisables. Il faut donc sécuriser le **cadre** lui-même :

- normaliser les entrées ;
- centraliser l'auth ;
- cloisonner les tenants ;
- contrôler les tailles de requêtes ;
- protéger les secrets ;
- gouverner les accès par rôles et politiques.

Le projet y répond de façon cohérente sur toute la chaîne.

7. Jeu d'Essai

La fonctionnalité la plus représentative du projet est le scénario suivant :

enregistrer une base externe, y créer un schéma générique, puis l'interroger via le query-router sans écrire d'endpoint spécifique.

Vue synthétique du scénario



Cas nominal

#	Données en entrée	Données attendues	Données obtenues	Écart
1	Enregistrement d'une base PostgreSQL externe valide	La base est stockée chiffrée dans <code>tenant_databases</code>	Conforme attendu	Aucun
2	Création d'une table <code>contacts</code> via <code>schema-service</code>	Table créée avec <code>id</code> , <code>owner_id</code> , timestamps, RLS	Conforme attendu	Aucun
3	Insertion via <code>query-router</code>	Ligne créée, <code>owner_id</code> injecté automatiquement	Conforme après correction	Aucun
4	Lecture par le même utilisateur	Retour des lignes propres à l'utilisateur	Conforme	Aucun
5	Lecture par un autre utilisateur	Aucune fuite de lignes	Conforme	Aucun

Cas d'erreur

#	Données en entrée	Données attendues	Données obtenues	Écart
6	Nom de table invalide	Rejet avec erreur 400	Conforme	Aucun
7	Type SQL non supporté dans le schéma	Rejet avec erreur 400	Conforme	Aucun

#	Données en entrée	Données attendues	Données obtenues	Écart
8	Action inconnue dans <code>query-router</code>	Rejet avec erreur 400	Conforme	Aucun
9	JWT absent	Refus d'accès	Conforme	Aucun
10	API key absente	Rejet par Kong	Conforme	Aucun

Analyse

Ce scénario démontre la valeur fondamentale du projet :

- la structure se crée sans code métier ;
- la sécurité n'est pas oubliée ;
- le moteur fonctionne pour différents projets ;
- l'exploitation reste unifiée.

8. Installation & Utilisation

8.1 Prérequis

Avant de démarrer :

- Docker Engine ;
- Docker Compose v2 ;
- Bash ;
- OpenSSL ;
- Node.js si l'on veut lancer les validations TypeScript hors conteneurs ;
- ports disponibles : `8000` , `5432` , `27017` et autres selon profils.

8.2 Installation

```
# 1. Cloner le dépôt
git clone <url-du-repo>
cd mini-baas-infra

# 2. Générer les variables d'environnement
bash scripts/generate-env.sh

# 3. (Optionnel) Installer les dépendances du monorepo NestJS
cd src
npm install
cd ..

# 4. Vérifier la configuration
bash scripts/validate-all.sh
```

8.3 Lancement du Projet

DÉMARRAGE DU SOCLE PRINCIPAL

```
make up
```

DÉMARRAGE AVEC SERVICES ADDITIONNELS

```
make all-full
```

VÉRIFICATION SANTÉ

```
make health
```

LOGS

```
make logs  
make logs SERVICE=kong
```

ARRÊT

```
make down
```

Ports et points d'accès usuels

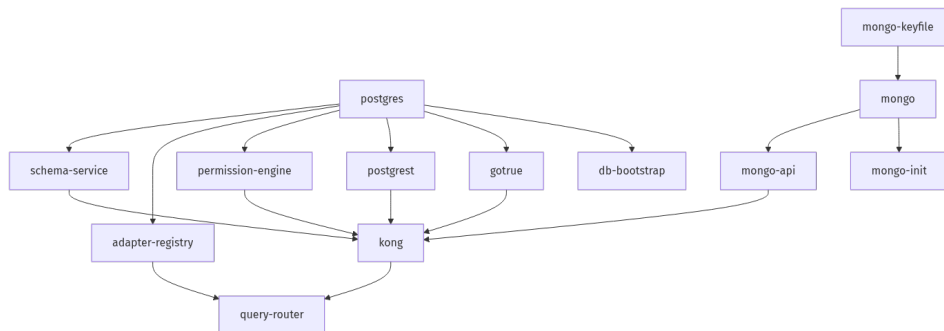
Service	URL / Port
Gateway	<code>http://localhost:8000</code>
PostgreSQL	<code>localhost:5432</code>
MongoDB	<code>localhost:27017</code>
Kong Admin	<code>localhost:8001</code>
Prometheus (profile observability)	<code>http://localhost:9090</code>
Grafana (profile observability)	<code>http://localhost:3030</code>
Studio (profile extras)	selon profil et configuration

Ordre de démarrage recommandé pour une démonstration

1. générer `.env` ;
2. `make up` ;
3. vérifier la santé ;
4. créer un compte via `/auth/v1` ;
5. tester PostgREST ou mongo-api ;
6. enregistrer une base externe ;
7. créer un schéma ;
8. exécuter des requêtes génériques.

8.4 Séquence de démarrage service par service

Cette sous-partie explicite le démarrage du système comme une chaîne de dépendances. Dans une architecture distribuée, l'ordre de disponibilité n'est pas un détail : il conditionne la reproductibilité de l'environnement et la stabilité du lancement.



ÉTAPE 1 — PRÉPARATION DES SECRETS ET VARIABLES D'ENVIRONNEMENT

Le système commence logiquement par la génération ou le chargement du fichier `.env`. Cette étape fixe :

- les mots de passe PostgreSQL ;
- le `JWT_SECRET` partagé ;
- les clés d'API Kong ;
- les secrets MinIO ;
- les paramètres MongoDB ;
- les tokens de service inter-services.

Sans cette couche, plusieurs conteneurs ne pourraient pas converger vers un état fonctionnel commun.

ÉTAPE 2 — DÉMARRAGE DES BASES DE PERSISTANCE PRIMAIRES

Les premières briques à démarrer sont `postgres` et `mongo` car elles supportent presque tous les autres services.

Rôle de `postgres` au démarrage :

- héberger les données de référence ;
- recevoir les rôles ;
- servir GoTrue, PostgREST, permission-engine et adapter-registry.

Rôle de `mongo` au démarrage :

- héberger les collections documentaires locales ;
- permettre le CRUD générique ;
- servir les change streams du realtime.

ÉTAPE 3 — JOBS DE BOOTSTRAP

Deux jobs transitoires sont critiques :

- `db-bootstrap` pour PostgreSQL ;
- `mongo-init` pour le replica set MongoDB.

`db-bootstrap` prépare notamment :

- le schéma `auth` ;
- les tables de démonstration ;
- les rôles `anon` , `authenticated` , `adapter_registry_role` ;
- les politiques RLS ;
- les tables de registre et de permissions.

`mongo-init` , lui, garantit que le replica set existe, condition importante pour certaines fonctionnalités temps réel.

ÉTAPE 4 — SERVICES D'IDENTITÉ ET DE PLAN DE DONNÉES INTERNE

Quand PostgreSQL est prêt et initialisé, plusieurs services peuvent démarrer :

- `gotrue` ;
- `postgrest` ;
- `permission-engine` ;
- `adapter-registry` ;
- `schema-service` .

Quand MongoDB est prêt :

- `mongo-api` devient disponible.

ÉTAPE 5 — GATEWAY ET EXPOSITION UNIFIÉE

Kong ne devient réellement utile qu'une fois ses upstreams clés joignables. Son démarrage tardif dans la chaîne évite d'exposer une façade vide ou incohérente.

ÉTAPE 6 — SERVICES DÉPENDANT DE LA GATEWAY OU D'AUTRES SERVICES INTERNES

Le `query-router` dépend à la fois :

- du registre d'adaptateurs ;
- de la disponibilité de la chaîne de routage ;
- du token de service.

Son positionnement en fin de séquence est cohérent.

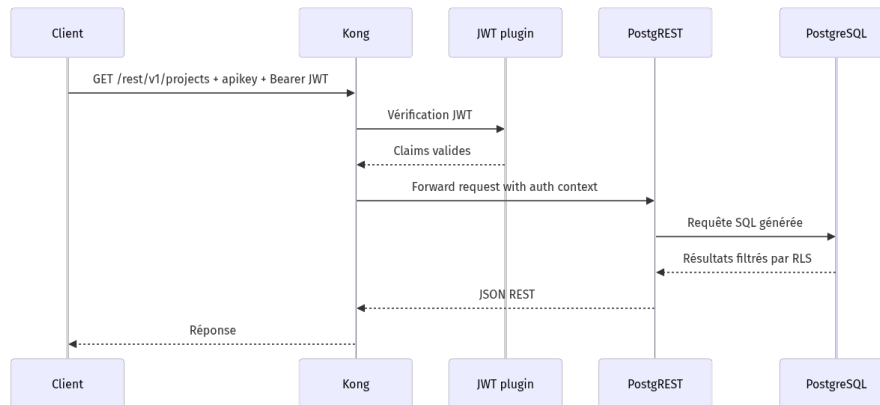
ÉTAPE 7 — SERVICES OPTIONNELS ET OBSERVABILITÉ

Les services `minio` , `storage-router` , `prometheus` , `grafana` , `loki` , `promtail` , `studio` , `pg-meta` , `supervisor` , `trino` s'ajoutent ensuite selon les profils activés.

8.5 Cycle complet des requêtes principales

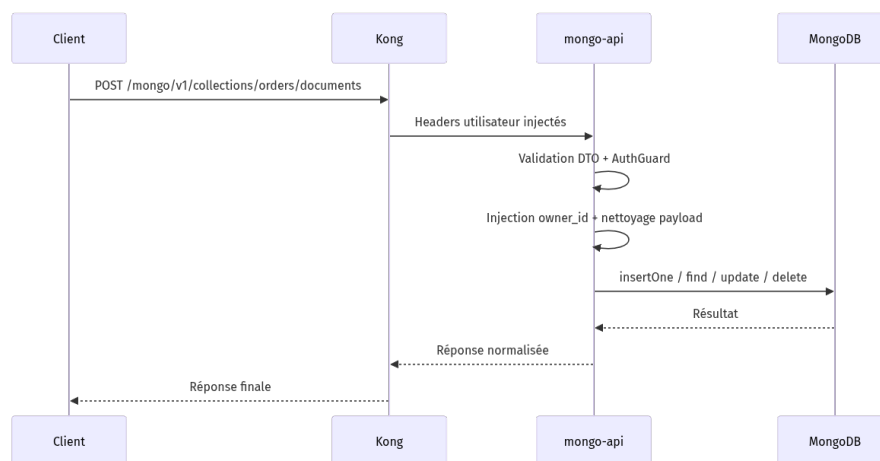
Cette section documente le comportement bout en bout pour les principaux flux techniques. Elle est particulièrement utile en soutenance pour montrer que l'on comprend non seulement les composants, mais aussi leur articulation dynamique.

8.5.1 CYCLE D'UNE REQUÊTE SQL VIA POSTGRES



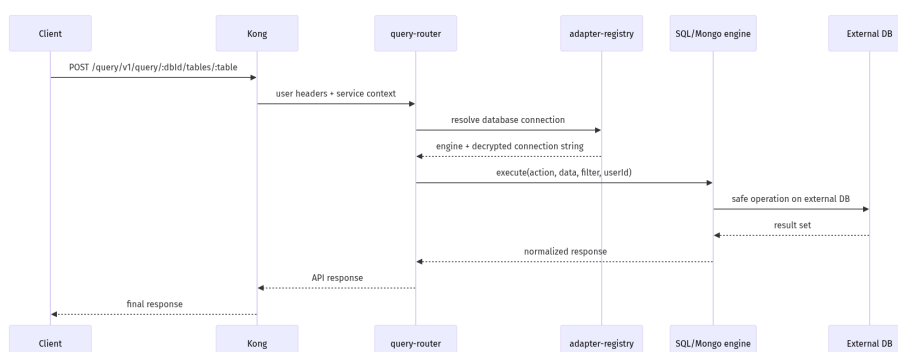
Analyse : ici, la logique applicative de base ne vit pas dans un contrôleur NestJS. Le couplage API-table est fourni par PostgREST, et la sécurité de ligne est portée par PostgreSQL.

8.5.2 CYCLE D'UNE REQUÊTE MONGODB VIA MONGO-API



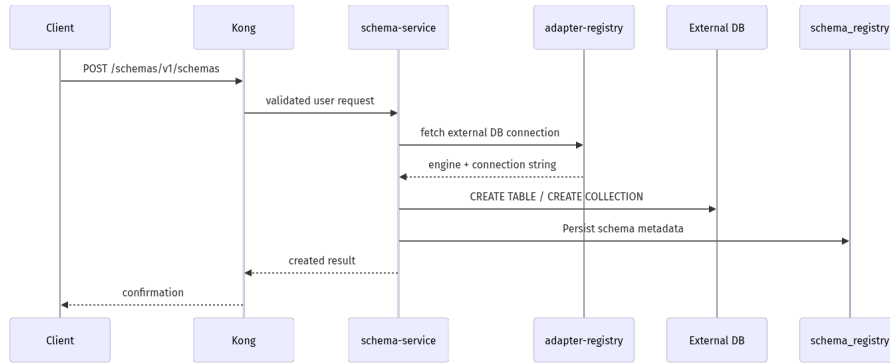
Analyse : ici, contrairement à PostgREST, la couche de protection propriétaire est assurée dans le service applicatif lui-même.

8.5.3 CYCLE D'UNE REQUÊTE SUR BASE EXTERNE VIA QUERY-ROUTER

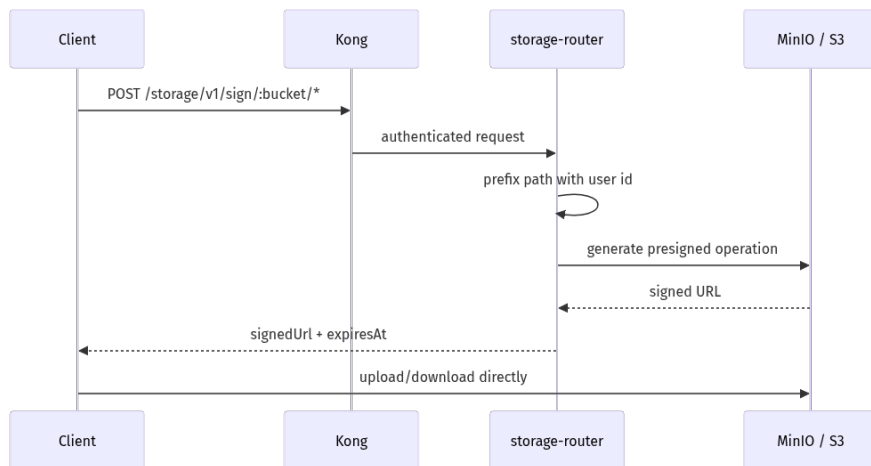


Analyse : ce flux est la démonstration la plus forte du caractère générique du projet, car il permet de manipuler une base distante sans créer de back-end dédié.

8.5.4 CYCLE DE CRÉATION DE SCHÉMA VIA **SCHEMA-SERVICE**

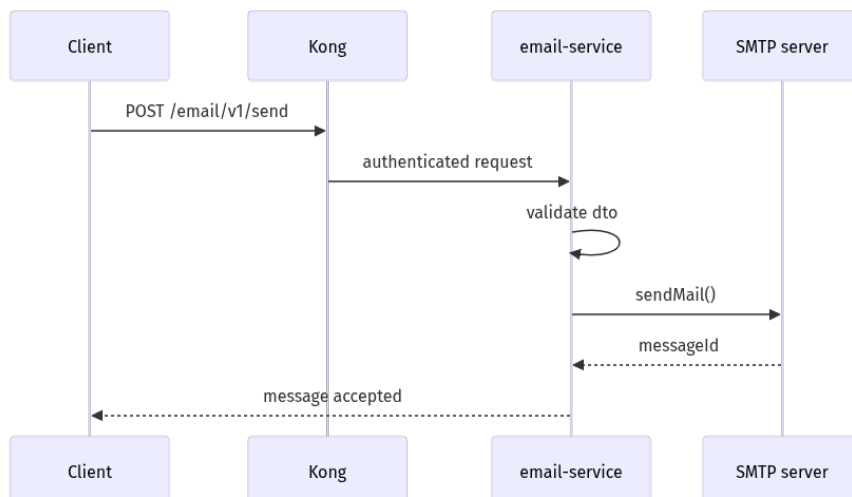


8.5.5 CYCLE DE STOCKAGE OBJET VIA **STORAGE-ROUTER**

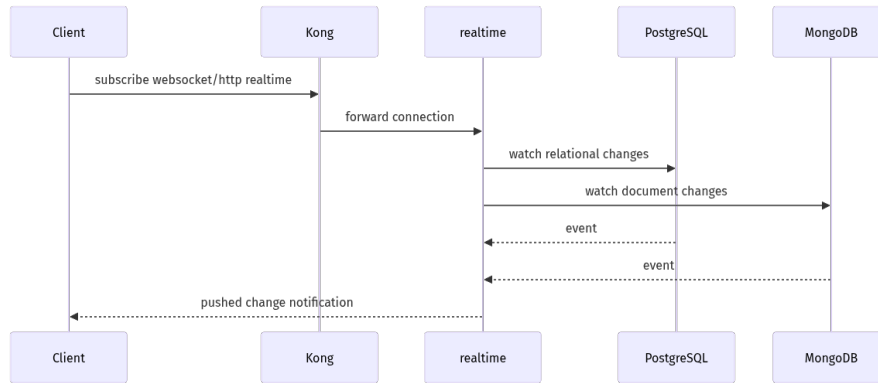


Analyse : le back-end ne transporte pas le fichier lui-même dans ce flux ; il délègue le transfert au stockage objet via une autorisation temporaire.

8.5.6 CYCLE D'ENVOI D'EMAIL VIA **EMAIL-SERVICE**



8.5.7 CYCLE TEMPS RÉEL



Analyse : ce flux transforme le backend en plateforme réactive, et non seulement transactionnelle.

9. Risques, Limites & Améliorations Futures

9.1 Risques techniques actuels

Risque	Description	Impact potentiel	Réponse actuelle / future
Complexité de pile	Multiplication des services à comprendre et exploiter	Courbe d'apprentissage plus forte	Documentation, Makefile, profils Compose
Couplage au gateway	Beaucoup de flux supposent Kong disponible	Dégradation globale si Kong tombe	Healthchecks, config déclarative, supervision
Gouvernance des bases externes	Une mauvaise base externe peut avoir des conventions inattendues	Difficulté de normalisation	<code>schema-service</code> , validation des types, registre central
Hétérogénéité SQL/Mongo	Deux modèles de données impliquent deux stratégies de sécurité	Complexité de raisonnement	Encapsulation par moteurs dédiés
Surface d'exposition générique	Un backend réutilisable expose des primitives puissantes	Risques d'abus si mal gouverné	API keys, JWT, rate limits, policies

9.2 Limites fonctionnelles

- absence d'interface d'administration unifiée dédiée aux services custom ;
- support limité volontairement à certaines opérations génériques ;
- dépendance aux conventions `owner_id` pour une partie des stratégies d'isolation ;
- pas de moteur GraphQL générique custom à ce stade ;
- pas de workflow low-code complet de gestion fonctionnelle ;
- pas de haute disponibilité multi-nœuds décrite dans ce dépôt.

9.3 Limites pédagogiques à expliciter à l'oral

Pour un exercice, il peut être utile de préciser que :

- la plateforme vise d'abord la démonstration d'architecture et d'intégration ;
- elle est déjà très riche, mais tout n'est pas industrialisé au niveau d'un SaaS mondial ;
- le but n'est pas de remplacer tous les développements métiers, mais de réduire massivement les développements répétitifs.

9.4 Améliorations futures envisageables

Axe	Amélioration
Gouvernance	UI dédiée pour <code>adapter-registry</code> , <code>schema-service</code> , <code>permission-engine</code>
Sécurité	rotation automatisée des secrets, audit trail plus poussé
Data plane	support d'autres moteurs ou d'opérations analytiques enrichies
Developer experience	SDK client officiel, templates front-end de consommation
Observabilité	corrélation distribuée complète et dashboards spécialisés
Déploiement	packaging Kubernetes / Helm / GitOps
Documentation	guides orientés par cas d'usage métier

9.5 Synthèse stratégique

Le principal compromis du projet est le suivant :

il accepte une complexité d'infrastructure plus élevée pour réduire de façon radicale la répétition des développements back-end sur les futurs projets.

Ce compromis est cohérent si la plateforme est pensée comme un investissement transversal.

10. Glossaire

Terme	Définition
BaaS	Backend as a Service, plateforme fournissant des briques back-end mutualisées
RLS	Row-Level Security, mécanisme PostgreSQL de sécurité ligne par ligne
DTO	Data Transfer Object, structure validée de données entrantes/sortantes
JWT	JSON Web Token, jeton d'authentification signé
API Gateway	Point d'entrée unique qui applique politiques, routage et sécurité
Tenant	Entité logique isolée dans un système multi-tenant
ABAC	Attribute-Based Access Control, autorisation basée sur des attributs
RBAC	Role-Based Access Control, autorisation basée sur des rôles

Terme	Définition
Presigned URL	URL temporaire signée donnant un accès limité à un objet de stockage
Change Stream	Flux d'événements de changements côté MongoDB
Declarative config	Configuration décrite dans un fichier plutôt que construite dynamiquement à la main
Monorepo	Dépôt unique contenant plusieurs applications et bibliothèques
Healthcheck	Vérification d'état d'un service par l'orchestrateur
Upstream	Service cible derrière la gateway
Owner isolation	Stratégie consistant à filtrer les données par propriétaire

11. Structure du Projet

```

mini-baas-infra/
|
├─ docker-compose.yml          # orchestration principale
├─ docker-bake.hcl             # build multi-targets
├─ Makefile                    # commandes d'exploitation
├─ README.md                   # documentation racine
├─ docs/                       # documentation détaillée
├─ config/                     # configs observabilité
├─ docker/                     # contrats et services Docker
spécifiques
├─ scripts/                    # bootstrap, tests, validations,
migrations
|
├─ src/                        # monorepo NestJS
|   ├─ Dockerfile              # Dockerfile unifié des apps
|   ├─ nest-cli.json           # définition monorepo
|   ├─ tsconfig.json           # config TypeScript
|   └─ apps/
|       ├─ adapter-registry/
|       ├─ query-router/
|       ├─ schema-service/
|       ├─ permission-engine/
|       ├─ mongo-api/
|       ├─ storage-router/
|       └─ email-service/
|   └─ libs/
|       ├─ common/
|       ├─ database/
|       └─ health/
└─ playground/                # sandbox éventuel de démonstration

```

Lecture de structure par responsabilités

Dossier	Responsabilité
<code>src/apps</code>	logique applicative des microservices custom
<code>src/libs</code>	composants réutilisables transverses
<code>docker/services</code>	services ou configurations d'infrastructure
<code>scripts</code>	automatisation et validation
<code>docs</code>	capitalisation documentaire
<code>config</code>	observabilité et provisioning

12. Cartographie des Fichiers Sources

Cette cartographie relie les principaux fichiers du dépôt à leur rôle architectural. Elle a une double utilité :

- faciliter la soutenance en montrant où vit chaque responsabilité ;
- accélérer la maintenance en donnant une lecture orientée architecture.

12.1 Cartographie des applications NestJS

Fichier	Rôle architectural
<code>src/apps/adapter-registry/src/main.ts</code>	bootstrap HTTP, logger, Swagger, pipeline global
<code>src/apps/adapter-registry/src/app.module.ts</code>	composition racine du service registre
<code>src/apps/adapter-registry/src/health.controller.ts</code>	santé du service
<code>src/apps/adapter-registry/src/crypto/crypto.module.ts</code>	exposition du composant de chiffrement
<code>src/apps/adapter-registry/src/crypto/crypto.service.ts</code>	chiffrement/déchiffrement AES-256-GCM
<code>src/apps/adapter-registry/src/databases/databases.module.ts</code>	module fonctionnel du registre
<code>src/apps/adapter-registry/src/databases/databases.controller.ts</code>	endpoints d'enregistrement / lecture des bases
<code>src/apps/adapter-registry/src/databases/databases.service.ts</code>	logique métier du registre et stockage chiffré
<code>src/apps/adapter-registry/src/databases/dto/register-database.dto.ts</code>	contrat de validation d'enregistrement
<code>src/apps/query-router/src/main.ts</code>	bootstrap du routeur de requêtes
<code>src/apps/query-router/src/app.module.ts</code>	assemblage du service

Fichier	Rôle architectural
<code>src/apps/query-router/src/health.controller.ts</code>	santé du query-router
<code>src/apps/query-router/src/query/query.module.ts</code>	module fonctionnel de requêtage
<code>src/apps/query-router/src/query/query.controller.ts</code>	endpoints d'exécution et de listing
<code>src/apps/query-router/src/query/query.service.ts</code>	orchestration du registre et des engines
<code>src/apps/query-router/src/query/dto/query.dto.ts</code>	contrat unifié des opérations de requêtage
<code>src/apps/query-router/src/engines/postgresql.engine.ts</code>	moteur PostgreSQL générique sécurisé
<code>src/apps/query-router/src/engines/mongodb.engine.ts</code>	moteur MongoDB générique sécurisé
<code>src/apps/schema-service/src/main.ts</code>	bootstrap du service de schéma
<code>src/apps/schema-service/src/app.module.ts</code>	assemblage racine
<code>src/apps/schema-service/src/health.controller.ts</code>	santé du service
<code>src/apps/schema-service/src/schemas/schemas.module.ts</code>	module métier de schémas
<code>src/apps/schema-service/src/schemas/schemas.controller.ts</code>	endpoints de création/listing/suppression
<code>src/apps/schema-service/src/schemas/schemas.service.ts</code>	orchestration DDL et registre
<code>src/apps/schema-service/src/schemas/dto/schema.dto.ts</code>	définition des schémas entrants
<code>src/apps/schema-service/src/engines/postgres-schema.engine.ts</code>	création/suppression de tables PostgreSQL
<code>src/apps/schema-service/src/engines/mongo-schema.engine.ts</code>	création/suppression de collections MongoDB
<code>src/apps/permission-engine/src/main.ts</code>	bootstrap du moteur de permissions
<code>src/apps/permission-engine/src/app.module.ts</code>	composition du service
<code>src/apps/permission-engine/src/health.controller.ts</code>	santé du service
<code>src/apps/permission-engine/src/permissions/permissions.module.ts</code>	module d'évaluation des permissions
<code>src/apps/permission-engine/src/permissions/permissions.controller.ts</code>	endpoints de vérification et de rôles
<code>src/apps/permission-engine/src/permissions/permissions.service.ts</code>	logique de vérification ABAC/RBAC
<code>src/apps/permission-engine/src/permissions/dto/permission.dto.ts</code>	DTO de check et d'affectation

Fichier	Rôle architectural
<code>src/apps/permission-engine/src/policies/policies.module.ts</code>	module de gestion des politiques
<code>src/apps/permission-engine/src/policies/policies.controller.ts</code>	endpoints CRUD de politiques
<code>src/apps/permission-engine/src/policies/policies.service.ts</code>	persistance et lecture des politiques
<code>src/apps/permission-engine/src/policies/dto/policy.dto.ts</code>	contrat des politiques
<code>src/apps/mongo-api/src/main.ts</code>	bootstrap du data plane Mongo
<code>src/apps/mongo-api/src/app.module.ts</code>	composition globale Mongo
<code>src/apps/mongo-api/src/health.controller.ts</code>	santé du service
<code>src/apps/mongo-api/src/collections/collections.module.ts</code>	module CRUD documentaire
<code>src/apps/mongo-api/src/collections/collections.controller.ts</code>	endpoints CRUD collections
<code>src/apps/mongo-api/src/collections/collections.service.ts</code>	logique owner-scoped Mongo
<code>src/apps/mongo-api/src/collections/dto/collection.dto.ts</code>	DTOs documentaires
<code>src/apps/mongo-api/src/admin/admin.module.ts</code>	module d'administration Mongo
<code>src/apps/mongo-api/src/admin/admin.controller.ts</code>	endpoints admin collection/schema/index
<code>src/apps/mongo-api/src/admin/admin.service.ts</code>	gestion validateurs et index
<code>src/apps/mongo-api/src/admin/dto/admin.dto.ts</code>	DTOs admin Mongo
<code>src/apps/storage-router/src/main.ts</code>	bootstrap du service de storage
<code>src/apps/storage-router/src/app.module.ts</code>	composition du service
<code>src/apps/storage-router/src/health.controller.ts</code>	santé du service
<code>src/apps/storage-router/src/storage/storage.module.ts</code>	module fonctionnel storage
<code>src/apps/storage-router/src/storage/storage.controller.ts</code>	endpoint de présignature
<code>src/apps/storage-router/src/storage/storage.service.ts</code>	génération des URLs signées
<code>src/apps/storage-router/src/storage/dto/presign.dto.ts</code>	contrat de présignature
<code>src/apps/email-service/src/main.ts</code>	bootstrap du service email

Fichier	Rôle architectural
<code>src/apps/email-service/src/app.module.ts</code>	composition racine
<code>src/apps/email-service/src/health.controller.ts</code>	santé du service
<code>src/apps/email-service/src/mail/mail.module.ts</code>	module email
<code>src/apps/email-service/src/mail/mail.controller.ts</code>	endpoint d'envoi
<code>src/apps/email-service/src/mail/mail.service.ts</code>	intégration SMTP
<code>src/apps/email-service/src/mail/dto/send-email.dto.ts</code>	contrat d'envoi d'email

12.2 Cartographie des bibliothèques partagées

Fichier	Rôle architectural
<code>src/libs/common/src/index.ts</code>	barrel export du socle commun
<code>src/libs/common/src/decorators/current-user.decorator.ts</code>	extraction du contexte utilisateur
<code>src/libs/common/src/guards/auth.guard.ts</code>	validation des headers utilisateurs injectés par Kong
<code>src/libs/common/src/guards/roles.guard.ts</code>	enforcement des rôles
<code>src/libs/common/src/guards/service-token.guard.ts</code>	auth inter-services ou fallback utilisateur
<code>src/libs/common/src/interfaces/user-context.interface.ts</code>	type commun du contexte utilisateur
<code>src/libs/common/src/config/env.validation.ts</code>	validation de configuration
<code>src/libs/common/src/filters/all-exceptions.filter.ts</code>	normalisation des erreurs
<code>src/libs/common/src/interceptors/correlation-id.interceptor.ts</code>	propagation des IDs de corrélation
<code>src/libs/common/src/pipes/validation.pipe.ts</code>	politique globale de validation DTO
<code>src/libs/database/src/index.ts</code>	exports du socle base de données
<code>src/libs/database/src/postgres/postgres.module.ts</code>	module PostgreSQL partagé
<code>src/libs/database/src/postgres/postgres.service.ts</code>	pools PostgreSQL, adminQuery, tenantQuery
<code>src/libs/database/src/mongo/mongo.module.ts</code>	module Mongo partagé
<code>src/libs/database/src/mongo/mongo.service.ts</code>	client Mongo mutualisé
<code>src/libs/health/src/index.ts</code>	exports santé
<code>src/libs/health/src/health.module.ts</code>	module de health dynamique
<code>src/libs/health/src/health.controller.ts</code>	endpoints live/ready standardisés

12.3 Cartographie des fichiers d'infrastructure majeurs

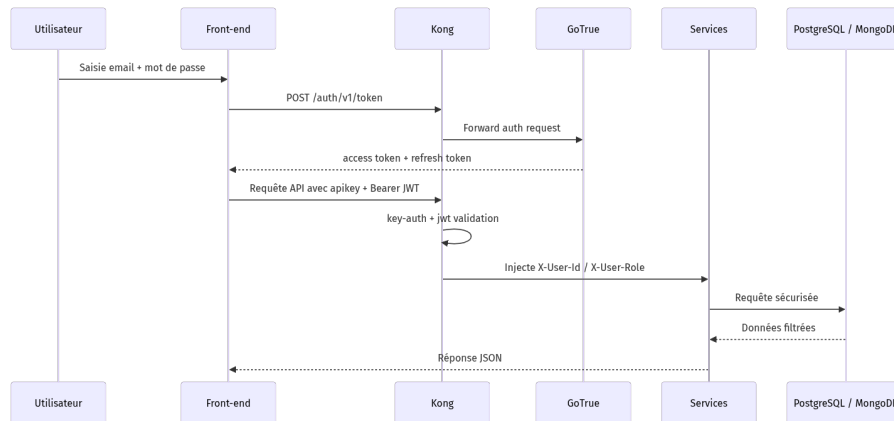
Fichier	Rôle architectural
<code>docker-compose.yml</code>	orchestration globale de la plateforme
<code>docker-bake.hcl</code>	stratégie de build parallèle multi-images
<code>Makefile</code>	façade d'exploitation développeur / ops
<code>scripts/db-bootstrap.psql</code>	initialisation structurante de PostgreSQL
<code>scripts/generate-env.sh</code>	génération sécurisée du <code>.env</code>
<code>scripts/validate-all.sh</code>	validation locale rapide avant exécution ou commit
<code>docker/services/kong/conf/kong.yml</code>	définition déclarative de la gateway
<code>src/Dockerfile</code>	build unifié des microservices NestJS
<code>src/nest-cli.json</code>	définition du monorepo Nest
<code>src/tsconfig.json</code>	base TypeScript commune

12.4 Cartographie documentaire et opérationnelle

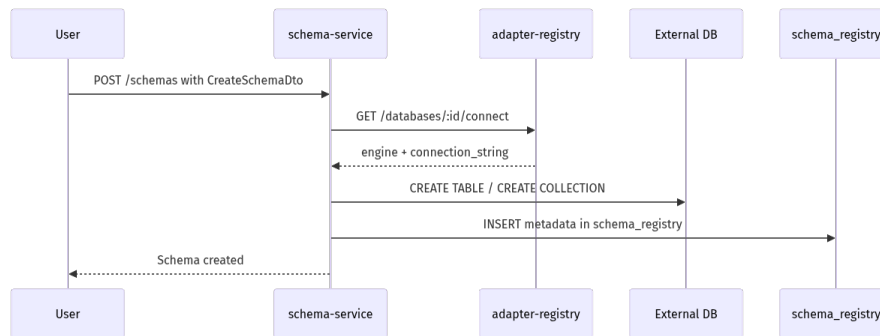
Fichier / dossier	Rôle architectural
<code>docs/README.md</code>	index documentaire
<code>docs/Infrastructure.md</code>	vue d'ensemble d'infrastructure
<code>docs/Kong-Gateway-Configuration.md</code>	détail du gateway
<code>docs/Kong-Database-Authentication-Integration.md</code>	chaîne auth/RLS
<code>docs/Mongo-Service-Validation.md</code>	validation du data plane Mongo
<code>docs/Partner-Demo-Runbook.md</code>	scénario de démonstration
<code>config/prometheus/</code>	configuration métriques
<code>config/grafana/</code>	provisioning dashboards
<code>config/loki/</code>	configuration logs
<code>config/promtail/</code>	collecte des logs Docker

13. Annexes

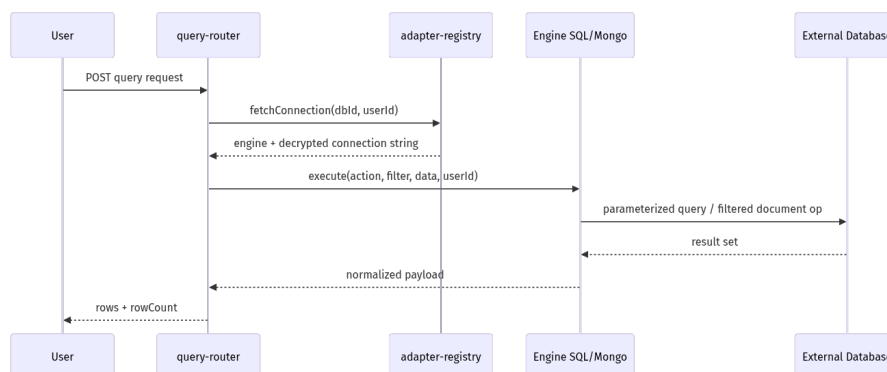
Annexe A – Diagramme complet du cycle d'authentification



Annexe B – Cycle de création d'un schéma externe



Annexe C – Cycle de requête universelle



Annexe D – Extrait de DTO unifié pour le query-router

```
export class ExecuteQueryDto {
  @IsEnum([
    "select",
    "insert",
    "update",
    "delete",
    "find",
    "insertOne",
    "updateMany",
    "deleteMany",
  ])
  action!: string;

  @IsOptional()
  @IsObject()
  data?: Record<string, unknown>;

  @IsOptional()
  @IsObject()
  filter?: Record<string, unknown>;
}
```

Ce DTO illustre la logique de standardisation : une enveloppe commune, plusieurs moteurs.

Annexe E – Pourquoi ce projet est une architecture back-end généraliste

Le projet mérite l'étiquette de **backend généraliste** pour les raisons suivantes :

1. il n'est pas lié à un métier particulier ;
2. il fournit des primitives universelles (auth, data, storage, email, realtime) ;
3. il peut être branché à plusieurs projets différents ;
4. il supporte plusieurs moteurs de persistance ;
5. il crée les schémas et interroge les bases sans endpoint spécifique ;
6. il applique des règles génériques de sécurité ;
7. il est industrialisable grâce à Docker Compose et à ses scripts.

Annexe F – Conclusion générale

Cette architecture n'est pas seulement une pile technique de services juxtaposés. C'est un **système cohérent**, pensé comme une usine à backends :

- le gateway protège et normalise ;
- l'authentification unifie l'identité ;
- les data planes exposent les données sans code métier répétitif ;
- le registre, le routeur de requêtes et le générateur de schémas rendent la plateforme adaptable à d'autres projets ;
- le moteur de permissions apporte la gouvernance ;
- les services transverses couvrent les besoins courants ;

- les outils d'observabilité et d'automatisation rendent l'ensemble exploitable.

En résumé, cette solution constitue une **fondation technique fortement réutilisable**, sécurisée et modulaire, capable de servir de back-end à de très nombreux projets sans repartir de zéro.

Document rédigé dans le cadre d'un exercice d'architecture et de documentation technique.

Version basée sur l'état du dépôt au 12 avril 2026.